SSN College of Engineering, Kalavakkam - 603110

Department of Information Technology

UG Project Work – Phase I – Review III

Bayesian Belief Network-Driven Proactive Fault Tolerance for Microservice Placement in Fog Computing Environments

Rahul Akshath Rajendran – 3122235002095

Mohit R – 3122235002072

Project Guide: Dr. Sabireen H, Assistant Professor

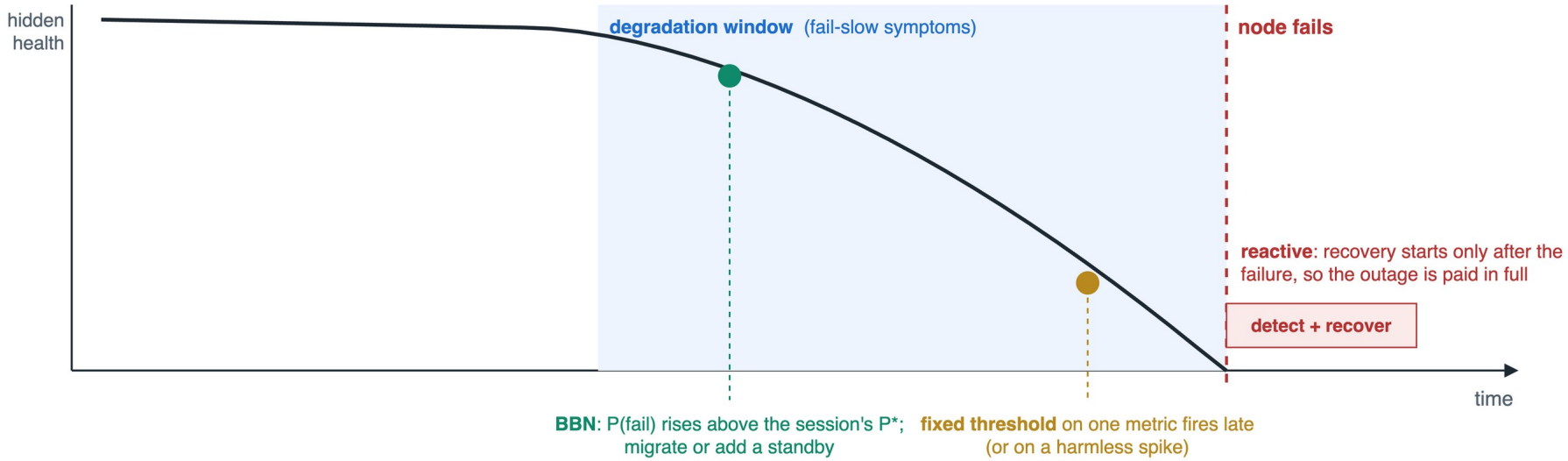
Outline

1. Problem Statement
2. Motivation and Research Gap
3. Proposed Work
4. Methodology
5. Dataset Description
6. Implementation
7. Demonstration
8. Experiments and Results
9. Inferences
10. Conclusion and Future Work

New since Review 2: dataset, implementation, demonstration and results (items 5–8)

Problem Statement

Fog nodes are fragile and usually **degrade before they fail**, yet fog fault tolerance is **reactive** (recovery starts after the failure) or **threshold-based** (a fixed cut-off on one metric). No existing mechanism estimates **the probability that a node fails within a given horizon**, so placement, migration and redundancy cannot be matched to the actual risk — and a silent telemetry channel, likely on a failing node, simply blinds them.



A proactive mechanism must act inside the degradation window, before the node fails

Motivation and Research Gap

Work	Mechanism	What it lacks
Rajab & Younis 2021	CPU-threshold checkpointing, primary-backup	fixed threshold; backup for every task
Syed et al. 2022	SDN priority routing, ACK retransmission	purely reactive; no replication
Naim et al. 2016	heartbeat polling, backup failover	detection-driven; single backup
Reyana et al. 2023	PSO-ABC placement optimisation	reliability only a static fitness term
Gupta & Singh 2025 (FTAPA)	health-index threshold migration	threshold hard to set; standby for all

The gap

- Node health is never treated as a **latent, uncertain** quantity
- No mechanism reasons when **telemetry is incomplete**
- No **calibrated probability** to weigh against the cost of acting
- Redundancy is **uniform**, not proportional to risk

Why a Bayesian Belief Network: it needs no large failure corpus (tables start from expert priors), sums out missing channels instead of imputing them, outputs a probability that can be multiplied by a cost, and names the condition behind each alarm.

Proposed Work

N1 BBN failure prediction

- P(node fails within 2 epochs | telemetry)
- missing channels summed out, never imputed
- names the condition behind the risk
- learned by MAP-EM, refined online

N2 Expected-loss migration and risk-aware placement

- migrate when P(fail) · C_fail > C_mig
- every session gets its own threshold
- placement utility over latency, energy and risk
- weights follow patient criticality

N3 Criticality-aware selective replication

- tier = least overhead + expected loss
- none · checkpoint · warm standby
- warm standbys capped at 20% of sessions
- recomputed every epoch

Since Review 2

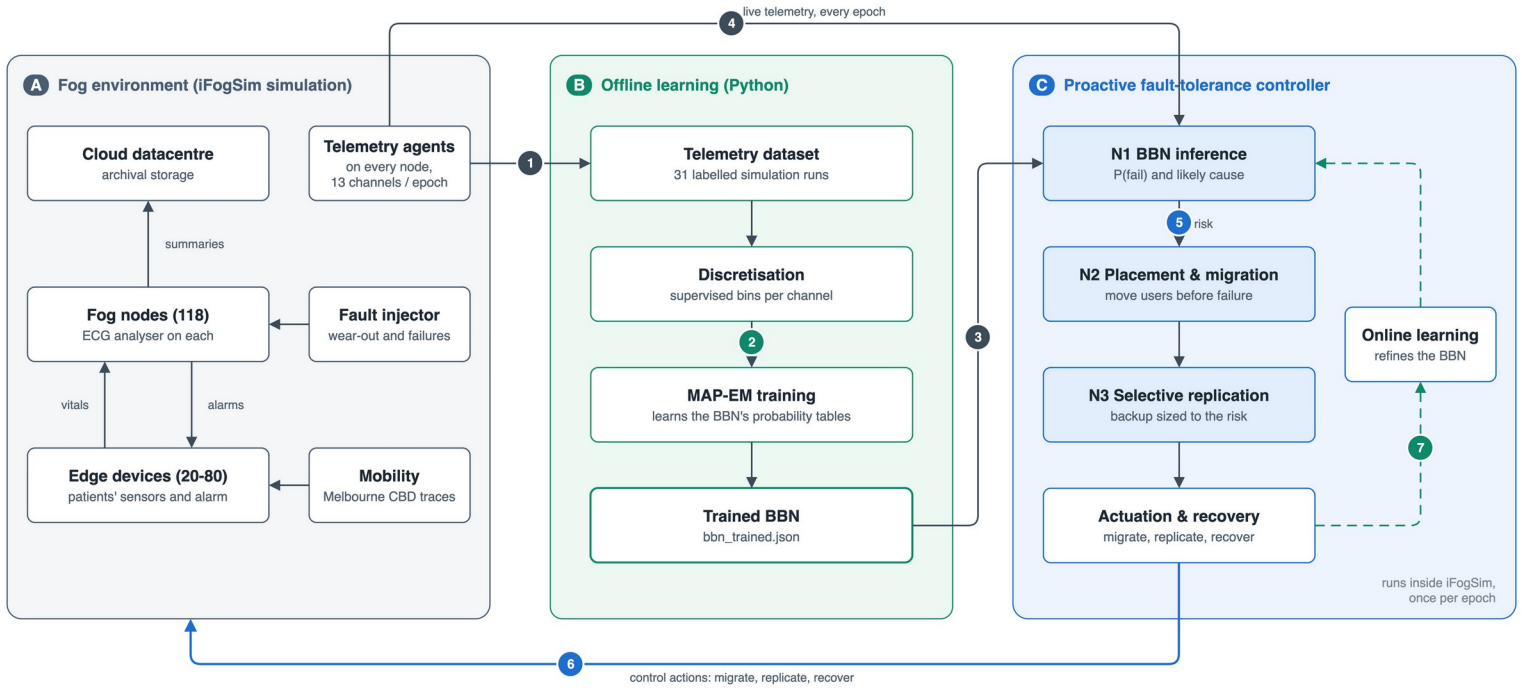
Valid data
degradation fault model; leak fixed (92.1% → 2.0%)

Stronger BBN
trend channels, device class, supervised bins, ranked nodes

Built in iFogSim2
controller with N1–N3 and six comparison policies

Evaluated
unseen regimes and seeds; paired statistical tests

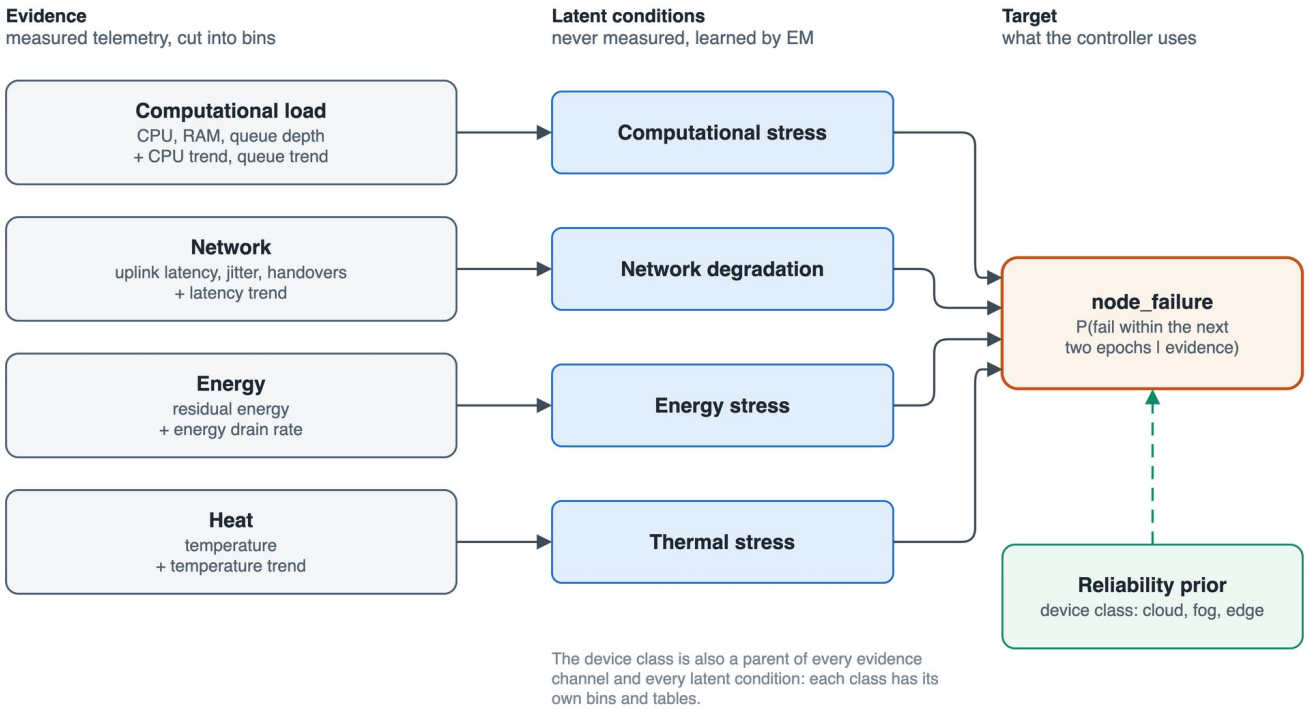
Methodology — System Block Diagram



1. devices report telemetry; runs are stored as a labelled dataset
2. telemetry is discretised and the BBN is trained by MAP-EM
3. the trained BBN is loaded by the controller
4. every epoch, live telemetry reaches N1, which computes P(fail)
5. N2 and N3 decide placement, migration and replication
6. actions are applied; failed nodes are detected and recovered
7. observed outcomes refine the BBN online

Legend: data flow · control actions · learning feedback

Methodology — The Bayesian Belief Network



Structure

- 13 evidence channels (8 from Review 2 + 5 trends) → 4 latent conditions → node_failure
- Device class conditions every channel and condition

Learning

- **Supervised bins:** ≤ 8 per channel and class, placed by exact DP on mutual information
- **Ranked-node tables:** 40 numbers per class instead of 32,768 rows
- **MAP-EM:** latent conditions never observed; 43,374 patterns, ≈ 10 s

Inference

- Exact; a dark channel is **summed out**, never imputed
- Cause = condition whose P(high) rose most above its prior

Methodology — Decision Rules

Placement (Eq. 4)

$$U(m, n) = \alpha(1 - \hat{L}) + \beta(1 - \hat{E}) + \gamma(1 - P(F_n = fail \mid e))$$

- $\hat{L}$: predicted loop latency / 150 ms deadline (with M/M/1 queueing)
- $\hat{E}$: access distance / 5 km (radio energy)
- (α, β, γ) = (0.30, 0.10, 0.60) critical … (0.60, 0.25, 0.15) best-effort

Migration (Eq. 5)

$$P(F_n = fail \mid e) \cdot C_fail(m) > C_mig(m, n \rightarrow n')$$

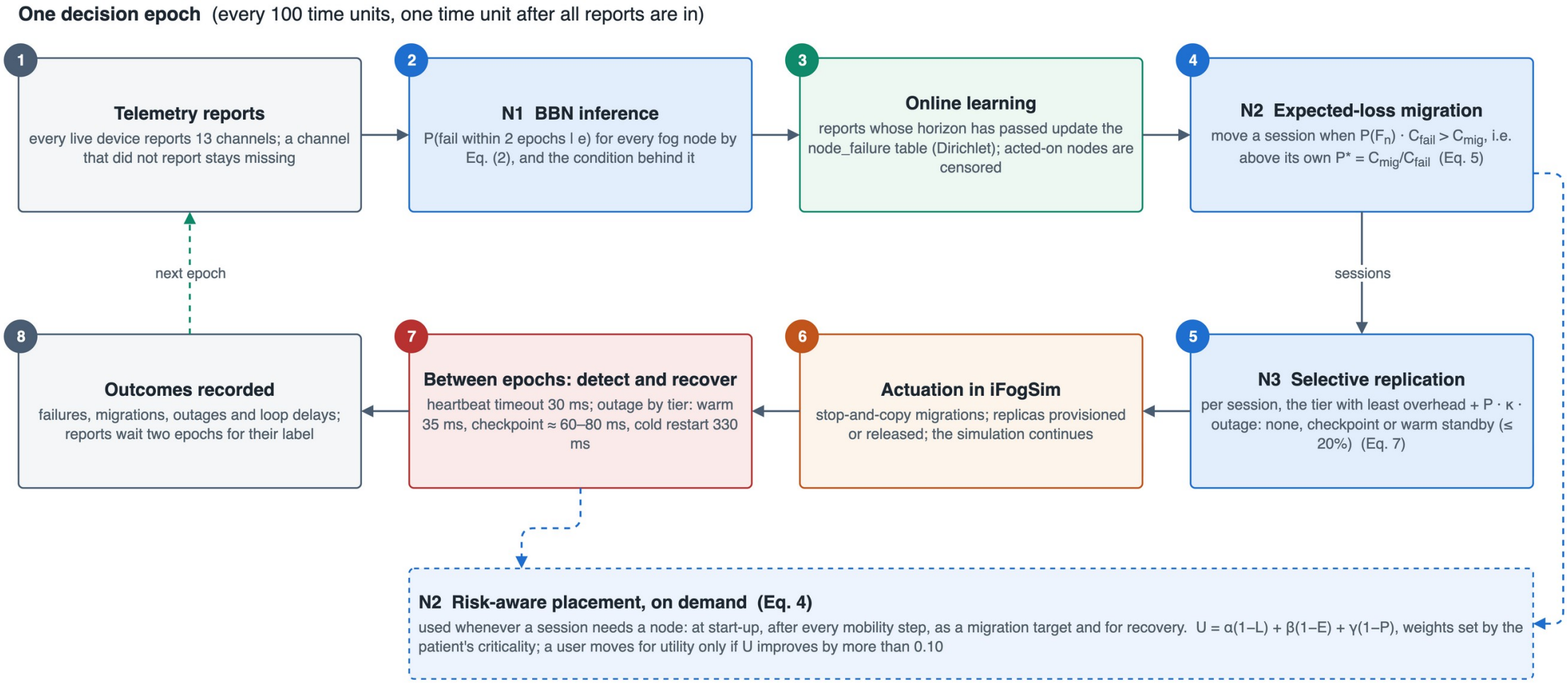
- each session's own threshold P* = C_mig / C_fail
- unprotected: P* = 0.020 critical, 0.022 standard, 0.049 best-effort
- no global threshold to tune

Replication (Eqs. 6–7)

$$\tau^*(m) = \operatorname{argmin} [\, O(\tau) + R(m) \cdot D(\tau) \,]$$
$$R(m) = P(F_n = fail \mid e) \cdot \kappa(m)$$

- outage: none 330 ms · checkpoint ≈ 60–80 ms · warm 35 ms
- critical patient: P = 0.001 → none; 0.05 → checkpoint; 0.30 → warm
- warm standby for at most 20% of sessions

Methodology — Execution Flow per Epoch



Dataset Description — Sources and Telemetry

Real inputs (shipped with iFogSim2)

- **Melbourne CBD edge resources:** 1 cloud + 118 fog gateways at real coordinates
- **Mobility traces:** 20–80 patients walk and hand over between gateways
- **Cardiac monitoring:** vitals → ECG analyser on the fog node → alarm; deadline 150 ms
- Patients: 25% critical, 45% standard, 30% best-effort

One record

- One device at one epoch (100 time units); 13 channels + device class
- **Label:** does the node fail within the next 200 time units?
- Hidden health and failure cause stored, **never used as features**

Channel	Feeds	From iFogSim2
cpu_util, ram_util, queue_depth	computational	host work, RAM, queues
cpu_trend, queue_trend	computational	agent's last 3 readings
uplink_latency, jitter_pktloss	network	link delay + load; arrivals
handover_rate, latency_trend	network	mobility; trend
residual_energy, energy_drain	energy	power model
temperature, temp_trend	thermal	fog/cloud only

Missing values stay missing: temperature on edge devices, jitter with fewer than three tuples, trends without history.

Dataset Description — Fault Model and Split



12 operating regimes; held-out runs use unseen regimes and unseen seeds

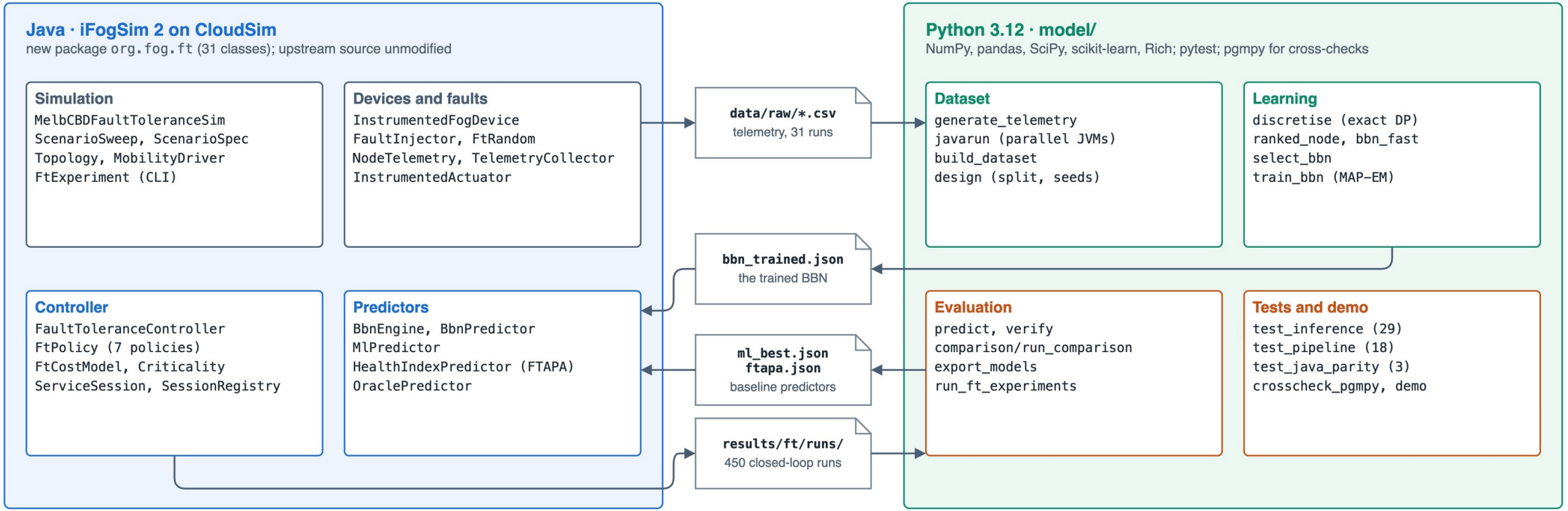
Fault model

- Hidden health wears faster under load, backlog, heat, energy depletion and poor links
- **Fail-slow symptoms** inside iFogSim: more CPU per tuple, slower links, more power and heat
- Wear-out, battery and **sudden** failures (≈ 1 in 5, no precursor)

Pool	Runs	Snapshots	Failing	Prevalence
Training	21	314,323	1,508	0.48%
Held out	10	160,823	422	0.26%

Leak audit: shared failures at the same epoch 92.1% (Review 2 injector) → **2.0%** now, i.e. chance level

Implementation — Technology Stack and Code



One model file, two engines. `bbn_trained.json` drives both the Python evaluation and the Java controller inside iFogSim; their posteriors agree to within 1e-15 (the parity test requires 1e-9), and the Python engine matches pgmpy.
One command each: `bash run_all.sh` regenerates every result (≈ 45 min); `bash run_demo.sh` runs the nine-stage demonstration (≈ 35 s).

Demonstration — bash run_demo.sh (1/3)

```
bash run_demo.sh — stage 5 of 9: live BBN training

5  Training the BBN on the devices' telemetry
node_failure  P(fail within 2 epochs | evidence)   <- 4 latent conditions + reliability prior
├── computational_stress  <- cpu_util, ram_util, queue_depth, cpu_trend, queue_trend
├── network_degradation   <- uplink_latency, jitter_pktloss, handover_rate, latency_trend
├── energy_stress         <- residual_energy, energy_drain
├── thermal_stress        <- temperature, temp_trend
└── reliability_prior     (device class: cloud / fog / edge)

  EM ──────────────────────────────────────────────────────── 60/60 iterations log-likelihood -0.00961
  ✓ 314,323 snapshots -> 43,374 distinct discretised patterns; trained in 10.6 s

Learned: P(fail within horizon | latent conditions), fog node
 computational      network     energy     thermal      P(fail)
 ──────────────────────────────────────────────────────────────
 low                high        low        low            0.000
 low                low         low        low            0.001
 low                low         low        high           0.008
 low                low         high       high           0.043
 low                high        low        high           0.242
 high               high        high       high           0.998
```

314,323 → 43,374

snapshots merged into distinct patterns

10.6 s

60 MAP-EM iterations, live

identical

retrained model = the one behind every result

Nine stages, ≈ 35 s

1. Build the iFogSim extension
2. The simulated system
3. Telemetry from the devices
4. Dataset and leak audit
5. **Live BBN training**
6. Prediction comparison
7. **Live BBN inside iFogSim**
8. **All policies, same run**
9. Verdict

Demonstration — BBN Controlling iFogSim Live (2/3)

```
stage 7 of 9: the trained BBN running fault tolerance inside iFogSim · scenario 7 (dense-edge, unseen) · seed 201

t = 9,901 / 10,000   fog nodes alive 113    at risk (P ≥ 0.1) 0    warm standby / checkpoint 0 / 5
proactive migrations 21  fog failures 5   users evacuated / hit 0 / 5    downtime so far 7.13 s
t= 9301.0  MIGRATE edge-59  (best-effort)   fog-61 → fog-127  P(fail)=0.410 > P*=0.272  cause: energy stress
t= 9301.0  MIGRATE edge-68  (best-effort)   fog-61 → fog-129  P(fail)=0.410 > P*=0.272  cause: energy stress
t= 9301.0  MIGRATE edge-73  (best-effort)   fog-61 → fog-129  P(fail)=0.410 > P*=0.272  cause: energy stress
t= 9313.9  FAIL    fog-38     (sudden  ) 4 user(s) hit
t= 9343.9  RECOVER edge-8   via none         → fog-22   outage 330 ms
t= 9343.9  RECOVER edge-28  via none         → fog-92   outage 330 ms
t= 9343.9  RECOVER edge-49  via none         → fog-92   outage 330 ms
t= 9343.9  RECOVER edge-74  via none         → fog-64   outage 330 ms
t= 9461.2  USER    edge-47 device died (sudden) -- outside fog control
t= 9701.0  MIGRATE edge-50  (standard   )   fog-61 → fog-127  P(fail)=0.372 > P*=0.111  cause: energy stress
t= 9701.0  MIGRATE edge-71  (best-effort)   fog-61 → fog-22   P(fail)=0.372 > P*=0.273  cause: energy stress
t= 9701.0  MIGRATE edge-75  (best-effort)   fog-61 → fog-29   P(fail)=0.372 > P*=0.049  cause: energy stress
t= 9901.0  MIGRATE edge-33  (standard   )  fog-127 → fog-50   P(fail)=0.033 > P*=0.022  cause: energy stress
t= 9901.0  MIGRATE edge-58  (standard   )  fog-127 → fog-97   P(fail)=0.033 > P*=0.022  cause: energy stress
online CPT updates: 18922 obs, 20 refreshes, max |dP| 0.0003

✓ run finished: availability 99.103%, 21 proactive migrations,
  0 users evacuated before a failure, 5 hit (5 by sudden failures, which have no precursor); controller 2.24 ms per epoch
```

Proactive migration

sessions leave fog-61 at P(fail) = 0.41 > P* = 0.272 — cause: energy stress

Own thresholds

same node, different P*: 0.111 standard vs 0.273 best-effort

Sudden failure

fog-38 has no precursor; 4 users recovered by cold restart (330 ms)

Demonstration — Every Policy on the Same Runs (3/3)

stage 8 of 9: every fault-tolerance policy on the same held-out runs

Policy	Failures+migr./user	BBN vs this	Wilcoxon p	Users hit	Warm standby	Crit. misses
No fault tolerance	517 ms	−97%	<0.0001	78.5	0.0%	3.82%
Cloud-only placement	2 ms	+863%	<0.0001	2.0	0.0%	100.00%
Reactive checkpoint + backup	111 ms	−85%	<0.0001	89.2	0.0%	4.39%
Threshold migration (FTAPA)	38 ms	−55%	0.0016	54.1	100.0%	5.46%
Expected-loss + ML predictor	19 ms	+18%	0.1219	3.8	0.3%	0.22%
Proposed BBN framework	20 ms			4.5	0.2%	0.13%
Oracle predictor (upper bound)	15 ms	+58%	0.0009	3.4	0.4%	0.12%

25 held-out runs per policy (5 unseen regimes x 5 unseen seeds), identical failure luck for every policy

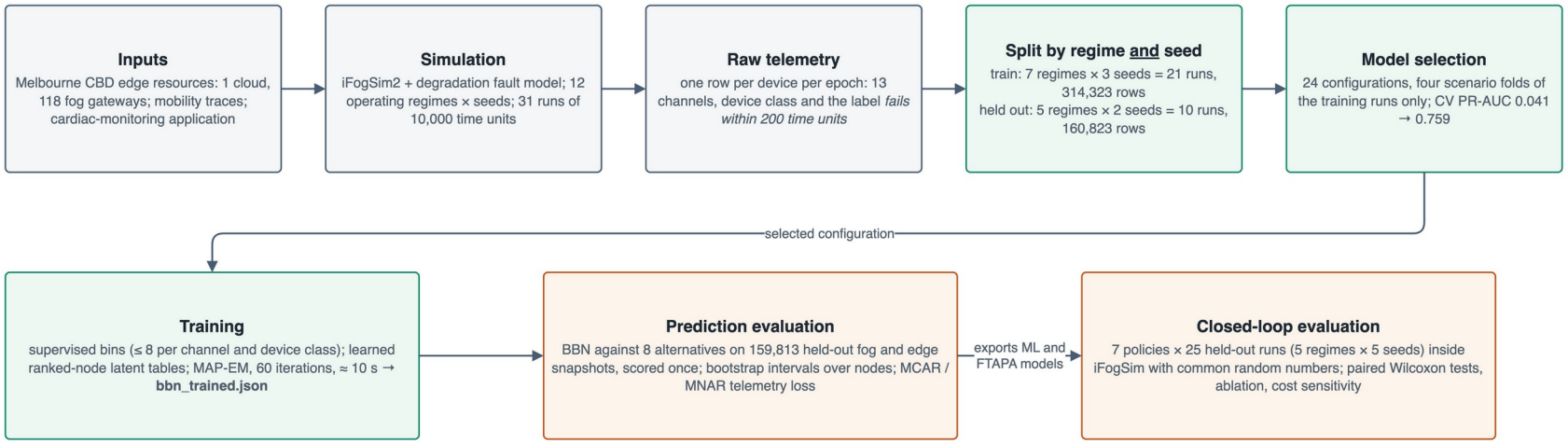
−97%
vs no fault tolerance

−55%
vs FTAPA threshold migration

0.2%
warm standby (FTAPA: 100%)

+18%
vs ML controller, p = 0.12 (n.s.)

Experiments — Protocol and Performance Metrics



Protocol

- choices on 4 scenario folds of training runs
- held-out pool scored once
- bootstrap CIs over nodes
- paired Wilcoxon tests, 25 runs

Prediction metrics

- ROC-AUC, PR-AUC (chance 0.0026)
- Brier, log-loss, ECE (calibration)
- precision, recall, F1; early warning
- robustness to MCAR / MNAR loss

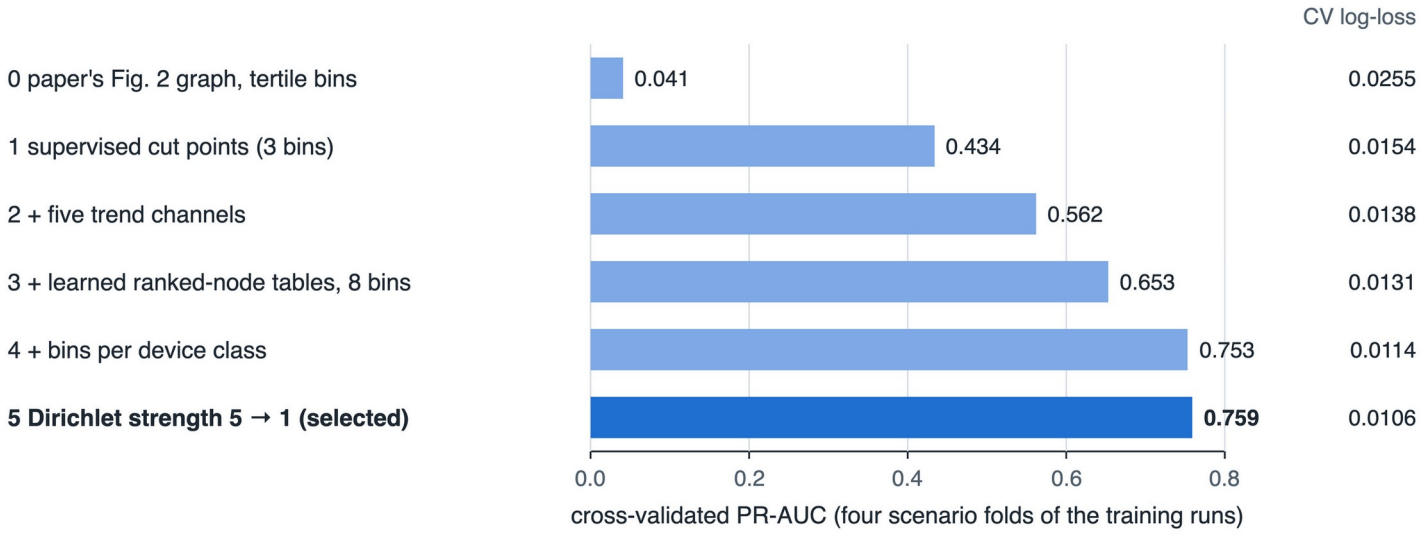
Service metrics

- downtime per user: failures + migrations
- users hit / evacuated; availability
- critical deadline misses (150 ms), p95
- warm-standby overhead

Baselines

- predictors: RF, GB, NB, LR, MLP, FTAPA, CPU
- policies: none, cloud-only, reactive
- FTAPA threshold migration
- ML-driven controller; oracle bound

Results — Improving the BBN (Training Folds Only)



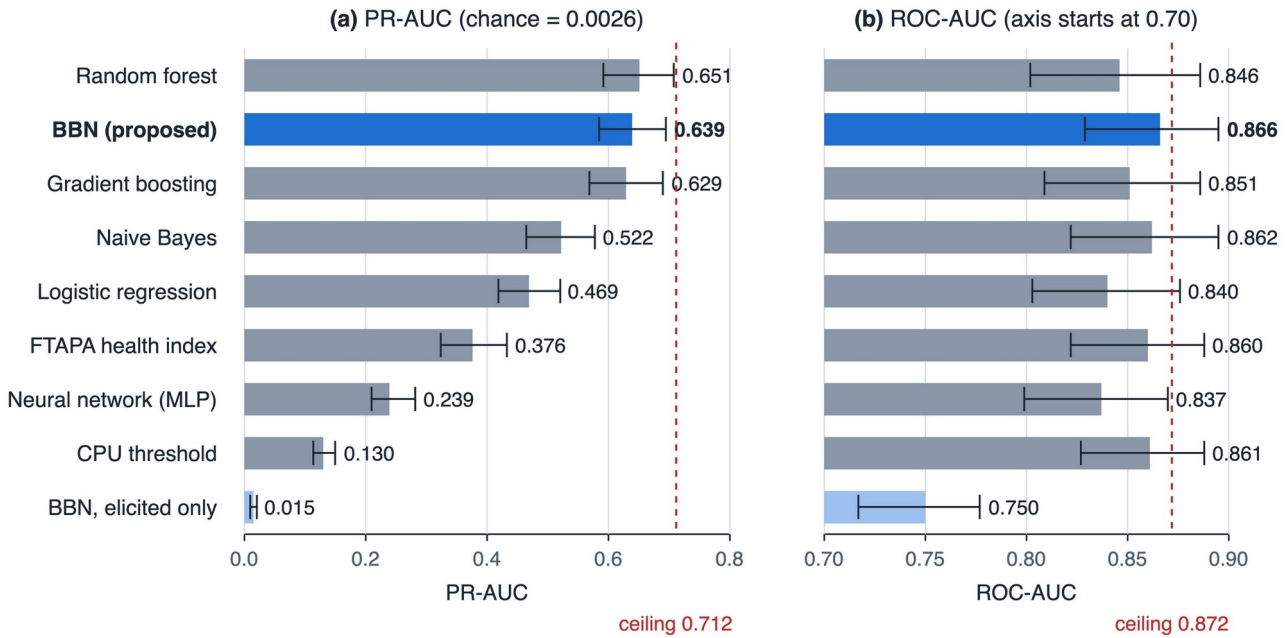
What moved accuracy

- Tertiles hide the tail where failures live: **0.041**
- Supervised cut points: **0.434**
- Trend channels catch drift: **0.562**
- Ranked nodes make 8 bins learnable: **0.653**
- Per-class bins: **0.753** → **0.759**

24 configurations, 4 scenario folds; the held-out pool was scored once, afterwards.

Then, once, on the held-out pool: ROC-AUC 0.866 · PR-AUC 0.639 · Brier 0.00128 · ECE 0.0005 — not directly comparable with the CV score: the training runs fail more often (0.48% vs 0.26%), and PR-AUC rises with prevalence

Results — Failure Prediction against Eight Models



0.866
ROC-AUC — best of 9 models

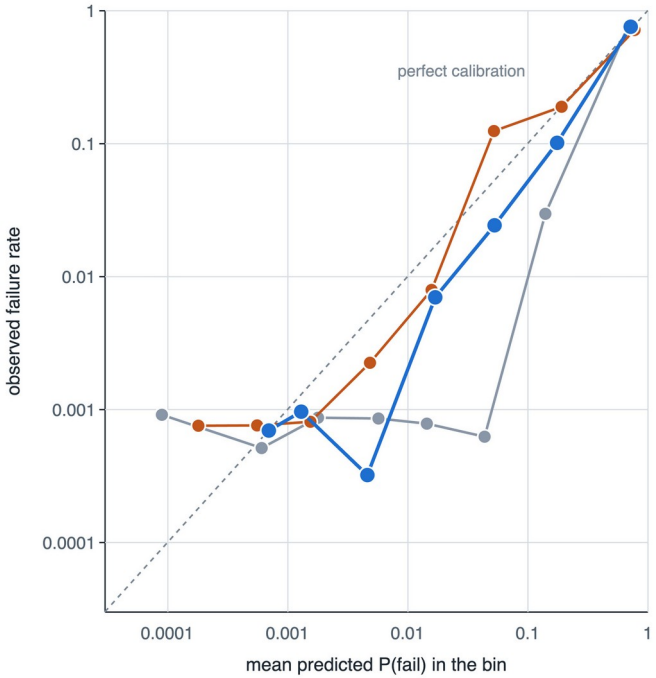
0.639
PR-AUC — tied with RF (0.651)

99% / 90%
of the achievable ROC / PR-AUC

63%
failing nodes flagged in time (RF 68%)

159,813 held-out snapshots, 422 failing (prevalence 0.26%); 95% bootstrap intervals over nodes

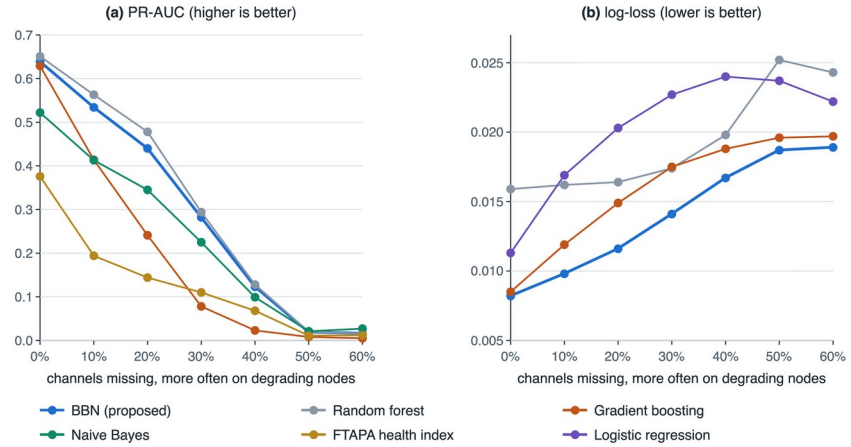
Results — Calibration and Missing Telemetry



Held-out pool

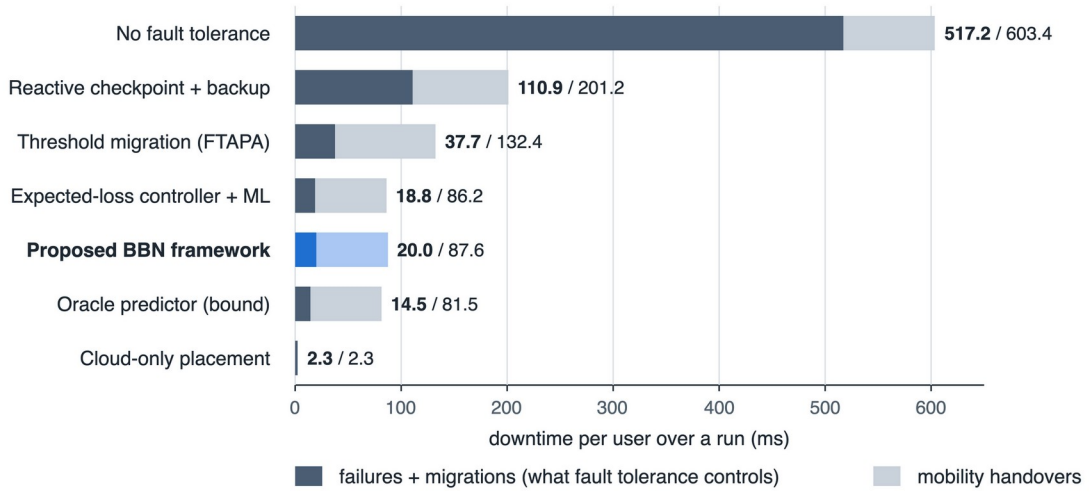
	Brier	ECE
BBN	0.00128	0.0005
Gradient boosting	0.00134	0.0006
Random forest	0.00157	0.0084

Bins by predicted probability (bins with ≥ 50 snapshots). Between 0.01 and 0.3 the BBN over-predicts by about a factor of two; the random forest puts 22,476 snapshots near 0.04 that fail at 0.0006, a seventy-fold gap.



- Best **Brier 0.00128** and **ECE 0.0005**: probabilities mean what they say — they are multiplied by costs
- Lowest **log-loss at every loss rate**; under heavy loss Naive Bayes ranks best (ROC 0.821 vs 0.700)

Results — Closed-Loop Fault Tolerance in iFogSim



	warm standby	critical deadline misses
No fault tolerance	0.0%	3.82%
Reactive checkpoint + backup	0.0%	4.39%
Threshold migration (FTAPA)	100.0%	5.46%
Expected-loss controller + ML	0.3%	0.22%
Proposed BBN framework	**0.2%**	**0.13%**
Oracle predictor (bound)	0.4%	0.12%
Cloud-only placement	0.0%	100.00%

−97 / −85 / −55%

fault downtime vs none, reactive, FTAPA

4.5

users hit per run (FTAPA 54.1)

0.13%

critical alarms past 150 ms

98.9%

of the oracle’s downtime reduction

Cloud-only placement: least downtime, but its 310 ms p95 loop delay misses every critical deadline

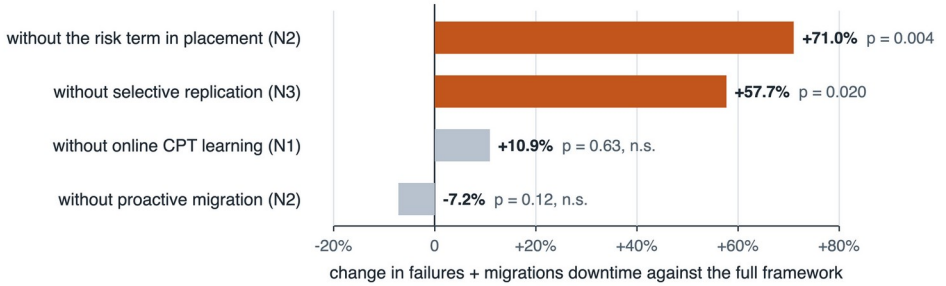
Results — Degraded Telemetry and Ablation

30% of channels missing on degrading nodes

Policy	Fault downtime / user	Total downtime / user
FTAPA threshold	33.9 ms	133.7 ms
Controller + ML	90.6 ms	156.2 ms
BBN framework	**65.9 ms**	**131.8 ms**

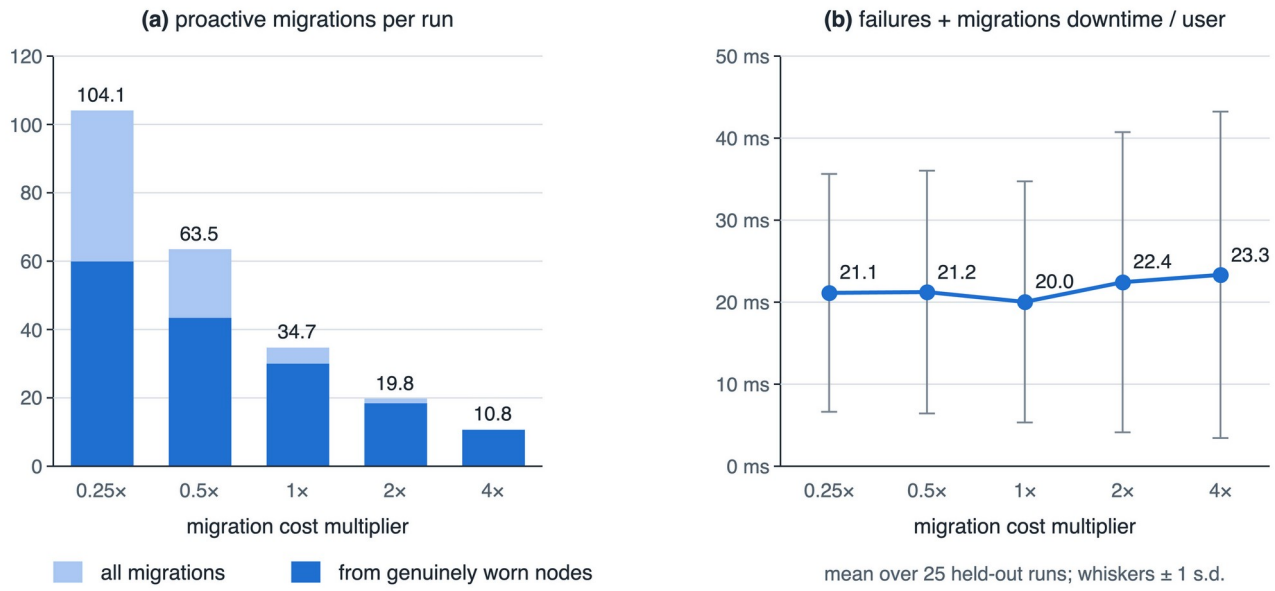
- BBN vs ML controller: **−29.2%** fault downtime (p = 0.006)
- Summing out a dark channel beats imputing a healthy-looking median
- FTAPA is most robust on fault downtime (p = 0.004) — bought with a standby for **every** session

Ablation: remove one component



- Risk-aware placement (+71%) and selective replication (+58%) are significant
- Online learning and migration alone: not significant once placement and replication are risk-aware

Results — Cost Sensitivity and Explanations



- Migrations fall **104 → 11** per run as the cost rises 16-fold
- Fault downtime stays **20.0–23.3 ms** per user: the rule calibrates itself

Does the named cause match?

Actual cause	Energy	Network	None
Battery (22)	**0.86**	0.14	0.00
Wear-out (278)	**0.92**	0.04	0.00
Sudden (122)	0.16	0.30	**0.52**

- Energy drain is the earliest wear symptom in this simulator
- Sudden failures have no precursor: mostly **no condition raised**
- Elicited tables alone (no failure data): ROC-AUC **0.750**

Inferences — Why a BBN, and Where It Is Not Ahead

Supported by the results

- Ranks failures as well as the best of 8 models (ROC-AUC best; PR-AUC tied)
- **Best-calibrated** probabilities — what cost-based rules need
- Summing out missing telemetry: **−29%** fault downtime vs ML under MNAR loss
- Beats thresholds with **0.2%** standby instead of 100%
- Usable before any failure is seen (elicited ROC-AUC 0.750)
- Explains every alarm with a named condition

Where it is not ahead

- Random forest: slightly higher PR-AUC (0.651), F1 (0.687), early detection (68%)
- Complete telemetry: ML-driven controller equal (+18%, $p = 0.12$)
- Heavy telemetry loss: every model collapses; Naive Bayes ranks best
- Sudden failures (29%) cannot be predicted — only redundancy helps
- Simulation study: a synthetic fault model, accelerated wear

Conclusion and Future Work

Conclusion

- A causal BBN, extended and learned by MAP-EM, predicts fog-node failure from incomplete telemetry and explains each alarm
- Inside iFogSim2 it drives placement, expected-loss migration and selective replication
- On unseen regimes and seeds: ROC-AUC 0.866 with the best calibration; fault downtime −97% / −85% / −55% vs none / reactive / FTAPA, with 0.2% warm standby
- Advantage over ML appears when telemetry fails with the node

Future work

- **Real failure traces** from fog / edge deployments, to test whether the learned precursors transfer
- **Dynamic Bayesian network** to model degradation over time and extend the horizon
- **Distributed inference** across fog clusters — no single controller to fail

References

[1] R. Mahmud, S. Pallewatta, M. Goudarzi, R. Buyya, “iFogSim2: An extended iFogSim simulator for mobility, clustering, and microservice management,” J. Systems and Software, vol. 190, 2022.

[2] H. T. Rajab, M. F. Younis, “Dynamic fault tolerance aware scheduling for healthcare system on fog computing,” Iraqi J. Science, pp. 308–318, 2021.

[3] S. A. Syed et al., “QoS aware and fault tolerance based software-defined vehicular networks using cloud-fog computing,” Sensors, vol. 22, no. 1, 2022.

[4] M. H. Naim, J. M. Zain, K. A. Jalil, “Fault tolerance mechanism for software application through fog computing as middleware,” IJCDS, vol. 5, no. 3, 2016.

[5] A. Reyana et al., “Opportunities of IoT in fog computing for high fault tolerance and sustainable energy optimization,” Sustainability, vol. 15, no. 11, 2023.

[6] S. Gupta, N. Singh, “Optimized fault-tolerant placement of microservices in distributed fog networks,” IJDIIC, vol. 4, no. 4, pp. 52–69, 2025.

[7] J. Pearl, Probabilistic Reasoning in Intelligent Systems. Morgan Kaufmann, 1988.

[8] N. E. Fenton, M. Neil, J. G. Caballero, “Using ranked nodes to model qualitative judgments in Bayesian networks,” IEEE TKDE, vol. 19, no. 10, 2007.

[9] A. P. Dempster, N. M. Laird, D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm,” J. Royal Stat. Soc. B, vol. 39, 1977.

[10] P. Weber, G. Medina-Oliva, C. Simon, B. Iung, “Overview on Bayesian networks applications for dependability, risk analysis and maintenance,” EAAI, vol. 25, no. 4, 2012.

[11] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, “On calibration of modern neural networks,” ICML, 2017.

[12] F. Wilcoxon, “Individual comparisons by ranking methods,” Biometrics Bulletin, vol. 1, no. 6, 1945.

Thank You